

Smart Shortlisting Engine: Technical Summary Report

A Robust, Evidence-First, 100% Local Multi-Signal Resume-to-Job Matching & Ranking System

Executive Summary: Built for the **Nexora AI Hackathon (InternLoom Challenge)**, this engine automates candidate shortlisting from batches of applicant resumes against complex Job Descriptions. Designed with strict compliance to **100% offline, zero-cloud API constraints**, the system avoids black-box LLM scoring in favor of a mathematically rigorous, dual-signal architecture combining deterministic lexical matching (exact, alias, fuzzy) and local dense semantic vector retrieval (all-MiniLM-L6-v2). It features an **Evidence-First Explainability Engine** that produces natural-language justifications with exact verbatim provenance for recruiter auditability, accompanied by a modern reactive web interface.

1. Problem Statement & Operational Constraints

The Core Recruitment Dilemma: Campus placement platforms like InternLoom evaluate hundreds of applicant resumes against each job posting before recruiter review. Traditional keyword search suffers from severe vocabulary mismatch (e.g. candidate writes "*REST APIs in Express & MongoDB*" while JD specifies "*Node.js Backend*"). Conversely, purely generative LLM approaches produce hallucinations, arbitrary non-reproducible scores, high cloud API latency, and data privacy leaks.

Constraint / Challenge	Engine Solution & Design Guarantee
100% Local & Offline	Zero external calls (OpenAI, Gemini, Claude). Pre-cached Sentence-Transformers running locally via PyTorch with HF_HUB_OFFLINE=1.
No Fake LLM Scoring	Scores calculated via closed-form hybrid math: lexical overlap (\$L\$), semantic embedding similarity (\$S\$), and requirement priority weighting.
Explainability Rubric (20%)	Deterministic natural language justifications citing exact resume section quotes, matched concepts, and explicit missing requirement gaps.
Messy & Corrupt Formatting	PyMuPDF engine handles multi-column layouts, missing headers, OCR noise, and gracefully isolates corrupted/zero-byte files without crashing.

2. System Architecture & Processing Pipeline

The pipeline converts raw unstructured PDF inputs into ranked shortlists through a synchronized 7-stage pipeline:

Pipeline Stage	Technology / Implementation Details	Key Output Artifact
1. Ingestion & Normalization	PyMuPDF multi-column text stream extractor, unicode normalizer, table & header detector.	Structured text blocks with line & page indices
2. Job Intelligence	Regex & pattern-based classifier separating hard mandatory constraints (REQUIRED) from soft bonus skills (PREFERRED).	JDProfile with categorized requirement entities
3. Candidate Profiling	Automated extraction of contact info, technical skills list, timeline experiences, educational credentials, and projects.	CandidateProfile with normalized sections
4. Dual Match Engine	• Lexical: Exact token match, technical alias dictionary (e.g. k8s → Kubernetes), and fuzzy Levenshtein (cutoff 0.85). • Semantic: 384-dim dense vectors from all-MiniLM-L6-v2 computing cosine similarity across structured resume paragraphs.	Paired lexical match spans & semantic cosine matrices (\$S\$, \$L\$)
5. Hybrid Scoring Engine	Closed-form dynamic scoring synthesis factoring match certainty, requirement importance, and semantic anti-hallucination floors.	Deterministic candidate score (\$0-100\$) + breakdown
6. Ranking & Tie-Break	Monotonic descending sort with tie-breaking rules prioritizing mandatory skill coverage, then experience seniority, then raw score.	Ordered candidate leaderboard (1..N)
7. Evidence Explainability	Evidence selector stitching verbatim quotes from resumes, identifying missing critical skills, and formulating recruiter comparisons.	Auditable CandidateExplanation narrative

3. Hybrid Scoring Engine & Mathematical Formulation

To eliminate non-deterministic or arbitrary ranking results, candidate scores are calculated through a multi-tiered formula combining normalized lexical score ($L \in [0, 1]$) and cosine semantic similarity ($S \in [0, 1]$):

1. Exact or Technical Alias Lexical Match ($L = 1.0$):

Score = $0.65 \cdot L + 0.35 \cdot S$ — Provides absolute credit for verified tool usage while rewarding deep contextual application.

2. Fuzzy Lexical Match ($0.85 \leq L < 1.0$):

Score = $0.50 \cdot L + 0.50 \cdot S$ — Equally weighs spelling variations against contextual semantic relevance.

3. Pure Conceptual Semantic Match ($L = 0.0$ and $S \geq 0.35$):

Score = $0.75 \cdot S$ — Credits strong conceptual overlap (e.g. Express → Node.js Backend). Scaled to 0.75 to prevent false equivalence.

4. Noise Rejection & Anti-Hallucination Threshold:

If $S < 0.35$, the semantic signal is truncated to 0.0 to prevent irrelevant text from accruing spurious points.

5. Requirement Priority Weighting:

Total Candidate Score = $\sum (\text{Weight}_{\text{req}} \cdot \text{Score}_{\text{req}}) / \sum \text{Weight}_{\text{req}}$ where $\text{Weight}_{\text{REQUIRED}} = 3.0$ and $\text{Weight}_{\text{PREFERRED}} = 1.0$.

4. Evidence-First Explainability & Bonus Features

A core requirement evaluated under the hackathon rubric is transparency: no candidate is assigned a ranking without clear evidence.

Feature / Innovation	Capability & Technical Implementation
Top-3 Candidate Explanations	Generates structured narrative briefs highlighting primary strengths, matched technologies with verbatim quotes, and specific missing requirements for each top applicant.
Pairwise Candidate Comparison	Addresses recruiter natural-language queries (e.g. 'Why is Candidate X ranked above Y?') by computing direct delta matrices on mandatory requirement coverage, semantic depth, and verified project experience.
Ranking Robustness Auditing	Integrated test harness subjecting resumes to formatting perturbations, synonym swaps, and token masks to confirm that ranking decisions remain stable and invariant to superficial formatting quirks.
Job Description Bias Detection	Scans JD text for restrictive phrasing, gender-skewed language patterns, or artificially narrow technology barriers that could unfairly depress applicant diversity.
Interactive Recruiter Web App	Local modern React 19 + Tailwind CSS dashboard providing batch upload, real-time progress indicators, shortlist cards, candidate drawer, and side-by-side comparative views.

5. Evaluation & Verification on Competition Dataset

The pipeline was validated against the official competition dataset (Junior Full Stack Developer Intern JD and 18 diverse candidate resumes) as well as an external audit suite of 54 PDF resumes:

Evaluation Metric / Test Category	Observed Result	System Assessment & Impact
Document Parse Success Rate	100% (18/18 official, 54/54 external)	Zero unhandled exceptions across all layout styles and multi-column formats.
Score Distribution & Dispersion	Clear score spread (Strong: 85-94, Medium: 60-78, Weak: 25-45)	Prevents score clustering; gives recruiters clean differentiation for shortlisting decisions.
Top-Ranked Profile Alignment	Top 3 applicants satisfy all REQUIRED criteria with verified evidence	Candidates with full-stack React/Node/PostgreSQL experience naturally rise to top ranks.
Corrupt & Zero-Byte Handling	Isolated in failed_candidates list	Corrupted files do not halt batch execution; transparent error logs displayed to user.
Inference Latency	~1.2 seconds per resume batch	Local CPU batch embedding generation enables rapid interactive recruiter workflow.

6. Technology Stack & Deliverable Artifacts

Layer	Technologies Used	Role in System
Backend Core	Python 3.9+, FastAPI, Uvicorn, Pydantic v2	High-throughput async REST API serving analysis and comparison endpoints
Parsing & NLP	PyMuPDF (fitz), Sentence-Transformers, PyTorch	PDF parsing, text extraction, offline embedding vectors (all-MiniLM-L6-v2)
Frontend UI	React 19, Vite, Tailwind CSS, Lucide Icons	Recruiter dashboard with batch upload, leaderboard, drawer, and compare view
Quality Assurance	Pytest, Playwright, Robustness Perturbation Audits	Automated unit, integration, and UI verification suites

Project Verification & Code Availability: All source code, test suites, architecture specifications, and evaluation datasets are version-controlled and public at GitHub: <https://github.com/DreamX55/Nexora-Brocode.git>.
Developed by Team Brocode for the Nexora AI Hackathon 2026 at Manipal Institute of Technology.